

1. ลำดับความซับซ้อน (Big O Complexity Hierarchy)

เขียนเรียงจาก เร็วไปช้า (ใช้น้อยไปมาก) เพื่อใช้เปรียบเทียบประสิทธิภาพ:

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$

- $O(1)$: คงที่ ไม่เพิ่มตามข้อมูล (เช่น การเข้าถึง Index ของ Array)
- $O(\log n)$: ลดขอบเขตข้อมูลครึ่งหนึ่งเสมอ (เช่น Binary Search)
- $O(n)$: ทำงานตามจำนวนข้อมูล (Loop ชั้นเดียว)
- $O(n^2)$: Nested Loop (Loop ซ้อนกัน 2 ชั้น)

2. วิธีการหา Big O จากโค้ด (Rules of Thumb)

ในการสอบ ให้มองหาโครงสร้างหลักๆ ดังนี้:

- การวนบท (Sequential):** ถ้าได้ทำต่อกัน ให้เอา Big O ตัวที่มากที่สุดมาตอบ
 - เช่น Loop $O(n)$ ต่อด้วย Loop $O(n^2)$ → ตอบ $O(n^2)$
- การวนซ้อน (Nested):** ถ้า Loop ซ้อนกัน ให้เอา Big O มาคูณกัน
 - เช่น for วั่ง n รอบ ซ้อนใน for ที่วิ่ง n รอบ → $O(n \times n) = O(n^2)$
- การข้ามค่า (Step):** * ถ้า $i = i + k$ หรือ $i = i - k \rightarrow O(n)$
 - ถ้า $i = i \times 2$ หรือ $i = i / 2 \rightarrow O(\log n)$

3. การประเมินประสิทธิภาพ (Case Analysis)

โจทย์อาจถามถึงสถานการณ์ที่ต่างกัน:

- Best Case (Ω - Omega):** กรณีที่อัลกอริทึมทำงานเร็วที่สุด (เช่น หาของเจอตัวแรกสุด)
- Average Case (θ - Theta):** กรณีเฉลี่ย (มักจะเท่ากับ Worst Case ในการเรียนพื้นฐาน)
- Worst Case (O - Big O):** กรณีที่แย่ที่สุด (ใช้เป็นมาตรฐานในการตอบข้อสอบเสมอ)

🔥 ลองฝึกทำ (ตัวอย่างโจทย์ประเมินประสิทธิภาพ)

โจทย์: จงหา Big O ของโค้ดต่อไปนี้

```
Python

def my_function(n):
    # ส่วนที่ 1
    for i in range(n):
        print("Hello")

    # ส่วนที่ 2
    i = 1
    while i < n:
        print("World")
        i = i * 3
```

วิธีคิด:

- ส่วนที่ 1:** เป็น Loop n รอบแบบตรงๆ → $O(n)$
- ส่วนที่ 2:** เป็น Loop ที่ค่า i เพิ่มขึ้นแบบคูณ 3 → $O(\log_3 n)$ ซึ่งใน Big O เราเขียนแค่ $O(\log n)$
- รวมกัน:** เนื่องจาก 2 ส่วนนี้ทำงานต่อกัน (Sequential) เราริงเอาตัวที่ หนักกว่า มาตอบ
- สรุป:** $O(n)$ ถ้าวัด $O(n)$ กับ $O(\log n) \rightarrow$ **ต้องเลือก $O(n)$**

🔖 ตัวอย่างที่ต้องจดลงกระดาษ (Cheat Sheet Code)

```
Python

# ตัวอย่าง O(n)
for i in range(n):
    print(i) # ทำงาน n ครั้ง

# ตัวอย่าง O(n^2)
for i in range(n):
    for j in range(n):
        print(i, j) # ทำงาน n * n ครั้ง

# ตัวอย่าง O(log n)
i = 1
while i < n:
    i = i * 2 # ค่า i เพิ่มขึ้นทีละ 2 เท่าให้ loop จบเร็วขึ้น
```

เทคนิคการเดา: หากเห็นโค้ดมีการ “แบ่งครึ่ง” ข้อมูล (เช่น `mid = (low + high) // 2`) ให้สงสัยไว้ก่อนว่าอาจมี $O(\log n)$ เข้ามาเกี่ยวข้องครับ

📊 ตารางสรุป Big O ของ Operation พื้นฐาน (จดไว้ได้ยิ่งดี!)

	Data Structure	Access	Search	Insert	Delete
Array/List		$O(1)$	$O(n)$	$O(n)$	$O(n)$
Linked List		$O(n)$	$O(n)$	$O(1)^*$	$O(1)^*$
Stack		-	-	$O(1)$	$O(1)$
Queue		-	-	$O(1)$	$O(1)$

1. รั้วติด Constant (ค่าคงที่)

Big O จะไม่สนใจส่วนคงที่ ให้ตัดออกให้หมด:

- $2n + 5 \rightarrow O(n)$
- $100n^2 + 50n \rightarrow O(n^2)$ (เฉพาะตัวที่ค่าสูงสุดสุด)
- ข้อระวัง: ถ้าใน Loop มี `print` 10 บรรทัด ก็ยังเป็น $O(1)$ ต่อรอบ ไม่ใช่ $O(10)$

2. เทคนิคการดู Loop ที่ซับซ้อน

- Loop ที่ลด/เพิ่มแบบทวีคูณ:**

```
Python

i = n
while i > 1:
    i = i // 2 # หรือ i = i / 2
```

🔥 **ตอบ:** $O(\log n)$ (เพราะจำนวนรอบจะเท่ากับจำนวนครั้งที่ n ถูกหาร 2 จนเหลือ 1)

- Loop ที่ค่าขอบเขต (Range) เปลี่ยนตาม Loop นก:**

```
Python

for i in range(n):
    for j in range(i): # # j รันตามค่า i
        print(j)
```

🔥 **ตอบ:** $O(n^2)$ (เพราะชั้นคือผลรวม $1 + 2 + 3... + n$ ซึ่งทางคณิตศาสตร์คือ $\frac{n(n+1)}{2}$)

Array / Python list 1 ข้อ · คุณลักษณะ · การดำเนินการ · การเข้าถึงข้อมูล

1. การคำนวณค่าพื้นที่ใน Memory (Address Calculation)

โจทย์มักถามว่า “ถ้า Array เริ่มที่ตำแหน่ง X ข้อมูลตัวที่ i จะอยู่ที่ตำแหน่งใด?”

- สูตร:** $Address(A[i]) = Base + (i \times size_of_element)$
 - Base:** ตำแหน่งเริ่มแรกของ Array (Index 0)
 - i:** Index ที่ต้องการหา
 - size:** ขนาดของข้อมูลแต่ละตัว (เช่น `int` = 4 bytes, `float` = 8 bytes)

2. การดำเนินการ (Operations) แบบเจาะลึก

คุณควรดูการจำลองการทำงาน เพื่อจะได้ Code ในข้อสอบ:

- Insertion (การแทรก):**
 - ต้องตรวจสอบก่อนว่า **Array เต็มไหม?** (Overflow)
 - ถ้าแทรกตรงกลาง ต้อง **Shift ข้อมูลไปขวา** (จากหลักลูกหมู) เพื่อเปิดที่ว่าง
 - ต้องดูขอบเขตในข้อว่าง
- Deletion (การลบ):**
 - ลบข้อมูลออก
 - ต้อง **Shift ข้อมูลไปทางซ้าย** (จากหน้าไปหลัง) เพื่อปิดช่องว่าง

3. ตารางเปรียบเทียบ Array ทั้งใน vs Python List

ข้อสอบมักถามความต่างเชิงคุณลักษณะครับ:

คุณสมบัติ	Array (static)	Python List (Dynamic)
ขนาด (Size)	คงที่ (Fixed) เปลี่ยนไม่ได้	ยืดหยุ่น (Dynamic) เพิ่ม/ลดได้
ชนิดข้อมูล	ต้องเหมือนกันหมด (Homogeneous)	ต่างชนิดกันได้ (Heterogeneous)
การเก็บข้อมูล	เก็บค่าข้อมูลลงในช่อง Memory โดยตรง	เก็บ Pointer ที่ชี้ไปยัง Object อื่นๆ
ความเร็ว	เร็วกว่า (ใช้ Memory น้อยกว่า)	ช้ากว่าเล็กน้อย (เพราะต้องอ่าน Pointer)

4. สรุป Time Complexity (ฉบับสมบูรณ์)

จดตารางนี้ไว้ที่มุมกระดานครับ:

- Access by Index:** $O(1)$
- Insert/Delete (at front):** $O(n)$
- Insert/Delete (at middle):** $O(n)$
- Insert/Delete (at end):** $O(1)$ (กรณี Python List)
- Search (Unsorted):** $O(n)$ (Linear Search)
- Search (Sorted):** $O(\log n)$ (Binary Search)

5. ความรู้เสริม: 2D Array (Row/Column Major)

ถ้ามีโจทย์ Array 2 มิติ (Matrix) การนับ Index จะเป็น:

- A[row][column]**
- การเก็บใน Memory มี 2 แบบ:
 - Row Major:** เก็บแถวที่ 0 ทั้งหมดก่อน แล้วค่อยขึ้นแถวที่ 1
 - Column Major:** เก็บหลักที่ 0 ทั้งหมดก่อน แล้วค่อยขึ้นหลักที่ 1

🔖 สรุปความต่างที่ต้องจด! (เปรียบเทียบ Array vs Linked List)

คุณสมบัติ	Array / List	Linked List
Memory	ต่อเนื่อง (Static/Dynamic)	ไม่ต่อเนื่อง (Dynamic)
Access (เข้าถึง)	$O(1)$ (ด้วย Index)	$O(n)$ (ต้องไล่จาก Head)
Insert/Delete	$O(n)$ (ต้องขยับข้อมูล)	$O(1)$ (แค่เปลี่ยน Pointer)*
พื้นที่	ใช้พอตกับข้อมูล	ใช้มากกว่า (ต้องเก็บ Pointer)

 ส่งออกไปยังฟีด



*หมายเหตุ: Insert/Delete ใน Linked List จะเป็น $O(1)$ เฉพาะเมื่อเรารู้ที่ตำแหน่งนั้นแล้ว แต่ถ้าต้องค้นหาตำแหน่งก่อนจะเป็น $O(n)$

3. Doubly Linked List (แบบสองทิศทาง)

- โครงสร้าง:** 1 Node มี 3 ส่วนคือ `Prev` | `Data` | `Next`
- ข้อดี:** เดินหน้าและย้อนหลังได้ ทำให้อ่าน Node ใดได้ง่ายกว่า Singly เพราะรู้ Node ก่อนหน้าทันที
- การดำเนินการ:**
 - Insertion/Deletion:** ซับซ้อนกว่าเพราะต้อง Update Pointer 2 ตัว (ให้ Next use Prev) ไม่เหมือน Singly ที่แก้แค่ตัว
 - การเชื่อมโยง:** ทำให้ได้ทั้งจากหน้าและหลัง $O(1)$ ถ้ามี Tail Pointer)

🔥 เทคนิคการทำข้อสอบ: การไล่ Pointer (สำคัญมาก!)

ในข้อสอบ 3 ข้อนี้ มักจะมีข้อหนึ่งให้ Code มาแล้วถามว่า “ผลลัพธ์คืออะไร” หรือ “Pointer จะชี้ไปที่ไหน”

- จำไว้:** `curr = curr.next` คือการเดินไปข้างหน้า 1 ตัว
- การแทนที่:** ต้องเขียน Pointer ของ “โหนดใหม่” เข้ากับ “โหนดหลัง” ก่อนแทนที่ (เพื่อให้ Address ของโหนดหลังหลุดออก) แล้วค่อยเปลี่ยน “โหนดหน้า” มาชี้ที่โหนดใหม่

Linked list 3 ข้อ · การดำเนินการ/ การนำไปใช้งาน · Simply LinkedList · Doubly LinkedList

1. Circular Linked List (แบบวงกลม)

เช่นเวอร์ชันที่ตัวสุดท้าย (Tail) ไม่ชี้ไปที่ `None` แต่ชี้กลับมาที่ `Head`

- จุดเด่น:** สามารถวนดูข้อมูลได้ไม่จบจนจากไหนก็ได้
- การใช้งาน:** ระบบคิวที่คำนวณค่าใหม่ หรือการจัดการคิวใน CPU (Round Robin Scheduling)
- สิ่งที่ต้องระวัง:** การเขียน Loop วนดูข้อมูลต้องเฝ้าเงื่อนไขให้ดี ไม่เช่นนั้นจะเกิด Infinite Loop

2. ลำดับขั้นตอนการ "แทรก" และ "ลบ" (Pointer Manipulation)

ข้อสอบมักให้ถามลำดับการเขียน Code (ข้อใดเรียงลำดับถูก) ให้จำกฎเหล็กคือ “**ทั้งต้นสายเก่าก่อนต้นสายใหม่**”

การแทรก Node ใหม่ (New) ระหว่าง A และ B:

- `A.next = A.next` (ให้ New ขึ้นที่ B ก่อน)
- `A.next = New` (แล้วค่อยให้ A ขึ้นที่ New)

ถ้าทำกลับกัน Address ของ B จะหายไปทันที (Memory Leak)

การลบ Node B ที่อยู่ระหว่าง A และ C:

- `A.next = B.next` (หรือ `A.next = A.next.next`)
- `del B` (ลบค่า Memory)

สำหรับ Doubly Linked List อย่าลืมอัปเดตตัว `prev` ของ C ให้กลับมาชี้ที่ A ด้วย

1. คุณลักษณะและการนำไปใช้งาน (Usage & Properties)

- คุณลักษณะ:** เก็บข้อมูลแบบไม่ต่อเนื่องใน Memory (Non-contiguous) แต่ตัวเราเรียกว่า **Node** เชื่อมกันด้วย **Pointer (Address)**
- Node ประกอบด้วย:** `Data` (ข้อมูล) และ `Next` (Address ของ Node ถัดไป)
- การนำไปใช้งาน:** * ใช้สร้าง Data Structure อื่นๆ เช่น Stack, Queue
- ใช้จัดการ Memory** ที่มีการเพิ่ม/ลดข้อมูลตลอดเวลา (Dynamic)
 - ใช้ในระบบที่ต้องการการแทรก/ลบข้อมูล “ตรงกลาง” ได้รวดเร็วถ้ามี Pointer อยู่แล้ว

2. Singly Linked List (แบบทิศทางเดียว)

- โครงสร้าง:** แต่ละ Node ขึ้นไปข้างหน้าอย่างเดียว ตัวสุดท้ายชี้ไปที่ `None` หรือ `Null`
- การดำเนินการ (Operations):**
 - Traversal (การทอดไป):** ต้องเริ่มจาก `Head` เสมอ วั่งไปจนเจอ `None` ($O(n)$)
 - Insertion (การแทรก):**
 - ที่หน้าสุด (Head):** สร้าง Node ใหม่ -> ให้ NewNode.next ขึ้นที่ Head เดิม -> เปลี่ยน Head มาที่ NewNode ($O(1)$)
 - ที่ท้ายสุด (Tail):** ต้องวนลูปไปจนเจอตัวสุดท้ายก่อน ($O(n)$) ยกเว้นมี Tail Pointer จะเป็น ($O(1)$)
 - Deletion (การลบ):**
 - ลบตัวหน้า:** เปลี่ยน Head ไปที่ Head.next ($O(1)$)
 - ลบตัวกลาง/ท้าย:** ต้องหาโหนดก่อนหน้า (Previous Node) แล้วข้ามโหนดที่จะลบไปชี้ตัวถัดไปแทน ($O(n)$ เพราะต้องงมหาตำแหน่ง)

🌟 การนำไปใช้งาน (Applications)

- ใช้ทำปุ่ม **Back / Forward** ใน Web Browser (Doubly Linked List)
- ใช้ทำระบบ **Image Viewer** หรือ **Music Player** ที่ความลับมาเพลงแรกได้ (Circular)
- ใช้จัดการ **Memory Allocation** ของระบบปฏิบัติการ

3. Sentinel Node (Header/Trailer Node)

บางครั้งโจทย์จะให้เราใส่ Node พิเศษขึ้นที่ข้อมูล (Dummy Node)

- Header Node:** อยู่หน้าจุดเชื่อม Head จริง
- Trailer Node:** อยู่หลังจุดเชื่อม Tail
- ประโยชน์:** ช่วยให้การเขียน Code ง่ายขึ้น เพราะไม่ต้องคอยเช็คเงื่อนไข `if head == None` (กรณี List ว่าง) หรือการแทรกตัวแรกสุด

1. การดำเนินการ (Operations)

- **Concept: LIFO** (Last-In, First-Out) — “เข้าทีหลัง ออกก่อน” เหมือนตั่งจานซ้อนกัน
- **ฟังก์ชันหลัก** (ทุกอย่างเป็น $O(1)$):
 - `push(x)` : เพิ่มข้อมูลลงที่ส่วนบนสุด (Top)
 - `pop()` : เอาข้อมูลด้านบนสุดออก (และคืนค่าตัวนั้น)
 - `peek()` หรือ `top()` : ดูค่าด้านบนสุดเฉยๆ โดยไม่เอาออก
 - `isEmpty()` : เช็คว่าง Stack ว่างไหม
- **ข้อระวัง:** * **Stack Overflow**: พยายาม Push ในขณะที่ Stack เต็ม
- **Stack Underflow**: พยายาม Pop ในขณะที่ Stack ว่าง

2. การแปลง Infix เป็น Postfix (สำคัญมาก!)

Infix: $A + B$ | Postfix: $AB+$ **กฎลำดับความสำคัญ (Priority):**

- วงเล็บ () (สำคัญสูงสุด)
- ยกกำลัง ^
- คูณ * และหาร /
- บวก + และ ลบ -

ขั้นตอนการแปลง (ใช้ Stack เก็บ Operator):

- เจอ **Operand** (ตัวเลข/ตัวแปร): พิมพ์ออกไปที่ผลลัพธ์ทันที
- เจอ (: Push ลง Stack ทันที
- เจอ) : Pop ทุกอย่างใน Stack ออกมาพิมพ์ จนกว่าจะเจอ ((แล้วเอา (ทิ้งไป)
- *เจอ **Operator** (+, -, *, /):
 - ถ้า Stack ว่าง หรือด้านบนสุดคือ (ให้ Push ลงไปเลย
 - ถ้าตัวที่จะลงไป **มีศักดิ์สูงกว่า** ด้านบนสุดใน Stack ให้ Push ลงไป
 - ถ้าตัวที่จะลงไป **มีศักดิ์น้อยกว่าหรือเท่ากัน** ด้านบนสุด ให้ Pop ตัวใน Stack ออกมาพิมพ์ก่อน แล้วค่อยเอีคใหม่/Push ตัวใหม่ลงไป

Queue 3 ข้อ · การดำเนินการ/ การนำไปใช้งาน

- **คิวเส้นตรง/คิววงกลม/คิวลำดับความสำคัญ**

1. การดำเนินการและการนำไปใช้งาน (Operations & Applications)

- **Concept: FIFO** (First-In, First-Out) — “เข้าก่อน ออกก่อน” เหมือนการต่อแถวซื้อของ
- **ตัวแปรสำคัญ:**
 - `Front` : ชี้ที่ตัวหน้าสุด (ตัวที่จะถูกเอาออก)
 - `Rear` : ชี้ที่ตัวสุดท้าย (ตัวที่เพิ่งเข้ามาล่าสุด)
- **ฟังก์ชันหลัก** (ทุกอย่างเป็น $O(1)$):
 - `enqueue(x)` : เพิ่มข้อมูลที่ส่วนท้าย (`Rear`)
 - `dequeue()` : เอาข้อมูลตัวหน้าสุด (`Front`) ออก
 - `isEmpty()` / `isFull()` : เช็คคิวว่างหรือเต็ม
- **การนำไปใช้งาน:** ระบบจองตั๋ว, การพิมพ์งาน (Printer Spooler), การส่งข้อมูลในเครือข่าย (Packets), และการจัดการคิวงานของ CPU

2. คิวเส้นตรง vs คิววงกลม (Linear vs Circular Queue)

คิวเส้นตรง (Linear Queue):

- **ปัญหา:** เมื่อ Dequeue ออกไปเรื่อยๆ พื้นที่ด้านหน้าจะว่างแต่ใช้งานไม่ได้ (เกิดอาการ “คิวเต็มหลอก” ทั้งที่มีที่ว่างข้างหน้า)
- **การแก้ปัญหา:** ต้องขยับข้อมูลทุกตัวมาข้างหน้า ($O(n)$) ซึ่งเสียเวลามาก

คิววงกลม (Circular Queue) (ต้องจดสูตรนี้!):

- **Concept:** ให้ตัวสุดท้ายวนกลับมาต่อที่ตัวแรกได้โดยใช้การ **Modulo (%)** กับขนาดของคิว (**MAX**)
- **สูตรการเลื่อนดังนี้:**
 - เลื่อน `Rear`: `Rear = (Rear + 1) % MAX`
 - เลื่อน `Front`: `Front = (Front + 1) % MAX`
- **การเช็คสถานะ:**
 - จำนวนข้อมูลในคิว: `count = (Rear - Front + MAX) % MAX`
 - คิวเต็ม: `(Rear + 1) % MAX == Front`

3. คิวลำดับความสำคัญ (Priority Queue)

- **Concept:** ข้อมูลแต่ละตัวจะมี “ค่าความสำคัญ” (Priority)กำกับอยู่ ตัวที่สำคัญกว่าจะได้ออกก่อนเสมอ (ไม่จำเป็นต้องเข้าก่อน)
- **การดำเนินการ:**
 - ถ้าความสำคัญเท่ากัน ให้ใช้หลัก FIFO (ตามลำดับเวลา)
 - **การนำไปใช้งาน:** ห่วงฉุกเฉิน (ER), ระบบประมวลผลงานใน OS ที่มีงานด่วน
- **ประสิทธิภาพ (Time Complexity):**
 - ถ้าใช้ Array/List ทัวไป: `Enqueue` $O(1)$ แต่ `Dequeue` ต้องหาตัวที่สำคัญสุดจะเสีย $O(n)$
 - ถ้าใช้ Heap (โครงสร้างต้นไม้): จะเป็น $O(\log n)$ ทั้งคู่

สรุปตารางเปรียบเทียบ Stack vs Queue

คุณสมบัติ	Stack	Queue
หลักการ	LIFO (เข้าทีหลัง ออกก่อน)	FIFO (เข้าก่อน ออกก่อน)
จุดเข้า/ออก	เข้าและออกที่ Top ทางเดียว	เข้าทาง Rear ออกทาง Front
ตัวอย่าง	ถาดอาหาร, ปุ่ม Undo	แถวจ่ายเงิน, คิวพิมพ์งาน

3. การคำนวณค่า Postfix (Evaluation)

ขั้นตอน (ใช้ Stack เก็บ Operand/ตัวเลข):

- เจอ**ตัวเลข**: Push ลง Stack
- เจอ **Operator**: * Pop ออกมา 2 ตัว (ตัวแรกทีออกมาคือ **ตัวขวา**, ตัวที่สองคือ **ตัวซ้าย**)
 - คำนวณ: (**ตัวซ้าย**) Operator (**ตัวขวา**)
 - Push ผลลัพธ์กลับลงใน Stack
- จบการทำงาน**: ตัวเลขที่เหลือใน Stack คือคำตอบสุดท้าย

ตัวอย่างจดลง A4 กันผาด

โจทย์: คำนวณ 5 3 2 * +

- เจอ 5 : Stack = [5]
- เจอ 3 : Stack = [5, 3]
- เจอ 2 : Stack = [5, 3, 2]
- เจอ * : Pop 2 (ขวา) และ 3 (ซ้าย) -> $3 * 2 = 6$ -> Push 6 -> Stack = [5, 6]
- เจอ + : Pop 6 (ขวา) และ 5 (ซ้าย) -> $5 + 6 = 11$ -> Push 11 -> Stack = [11] **คำตอบคือ 11**

✳ การนำไปใช้งาน (Applications)

- **Undo/Redo** ในโปรแกรมพิมพ์งาน
- **Function Call (Recursion)**: ระบบใช้ Stack เก็บสถานะการทำงาน (Call Stack)
- **Backtracking**: เช่น การหาทางออกจากเขาวงกต
- **Matching Parentheses**: ตรวจสอบว่าวงเล็บเปิด-ปิดครบและถูกลำดับหรือไม่

สรุป

1. Analysis of Algorithms (Big O)

- **ลำดับความเร็ว:** $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$
- **การดูโค้ด:**
 - `Loop` ขึ้นเดียว = $O(n)$ | `Loop` ซ้อนกัน = $O(n^2)$
 - แบ่งครึ่งข้อมูล/ทวีคูณ (เช่น `i = i*2`) = $O(\log n)$
 - คำสั่งพื้นฐาน (Assign ค่า, Print) = $O(1)$
- **กฎ:** ตัดค่าคงที่ทิ้ง ($2n + 5 \rightarrow n$) และเลือกตัวที่ “ช้าที่สุด” มาตอบเพียงตัวเดียว

2. Array / Python List

- **คุณลักษณะ:** เข้าถึงเร็ว ($O(1)$) แต่แทรก/ลบช้า ($O(n)$) เพราะต้อง **Shift** ข้อมูล
- **Python List:** เป็น Dynamic Array (ขยายขนาดได้), เก็บข้อมูลต่างชนิดกันได้
- **สูตร Address:** $Address(A[i]) = Base + (i \times size)$

3. Linked List (3 ข้อ - เน้นมาก)

- **Singly:** [Data | Next] -> ขึ้นทางเดียว, ตัวท้ายชี้ None
- **Doubly:** [Prev | Data | Next] -> เดินหน้า-ถอยหลังได้
- **หัวใจการแทรก (Insert):** ต้องเชื่อมต่อโหนดใหม่ไปยังทั้งก่อนและต่อ!
- 1. `newNode.next = curr.next`

2. `curr.next = newNode`
- **เปรียบเทียบ:** Array เข้าถึงเร็วกว่า ($O(1)$ vs $O(n)$) แต่ Linked List แทรก/ลบเร็วกว่าถ้าอยู่จุดตำแหน่งแล้ว

4. Stack (2 ข้อ)

- **Concept:** LIFO (Last-In, First-Out) | **เข้า/ออกที่ Top** ทางเดียว
- **Infix to Postfix:**
 - เจอ Operand (เลข) -> พิมพ์ออก
 - เจอ Operator -> ถ้าใน Stack ศักดิ์สูงกว่าหรือเท่ากัน ให้ Pop ออกมาก่อน แล้วค่อย Push ตัวใหม่ลงไป
 - **Priority:** `^ > *, / > +, -`
- **Postfix Eval:** เจอเลข Push ลง Stack, เจอ Operator ให้ Pop 2 ตัวมาคำนวณ (ตัวแรกทีออกคือตัวขวา)

5. Queue (3 ข้อ - เน้นสูตร)

- **Concept:** FIFO (First-In, First-Out) | **เข้าที่ Rear, ออกที่ Front**
- **Circular Queue (คิววงกลม):** ป้องกันคิวเต็มหลอก
 - เลื่อนตำแหน่ง: `(Index + 1) % MAX`
 - เช็คคิวเต็ม: `(Rear + 1) % MAX == Front`
- **Priority Queue:** ตัวที่สำคัญกว่าได้ออกก่อน ไม่สนลำดับการเข้า

ตารางสรุป Time Complexity (ต้องจด!)

Operation	Array / List	Linked List	Stack / Queue
Access (Index)	$O(1)$	$O(n)$	-
Search (Value)	$O(n)$	$O(n)$	-
Insert / Delete	$O(n)$	$O(1)^*$	$O(1)$

🗒 ส่องออกใบยังติด

*Linked List เป็น $O(1)$ เมื่อมี Pointer ชี้ที่ตำแหน่งนั้นแล้ว